

D-Vision-3D-Reconstructure

Memoir3D · Project Proposal & Build Roadmap

Prepared by Dana Khaing · June 06, 2026

Project Overview

D-Vision-3D-Reconstructure (product name: **Memoir3D**) transforms crowd-sourced event photos — taken by multiple attendees with different phones, angles, lighting, and timestamps — into a single navigable 3D scene with a timeline scrubber. The user can fly around the venue in a browser and scrub through time to relive any moment of the party.

Market gap: No existing tool (Luma AI, Polycam, Scaniverse) handles crowd-sourced multi-attendee photos with a temporal layer. The research foundation (WildGaussians NeurIPS 2024, CSS arXiv 2409.08562) is mature enough to build on.

Definitive Tech Stack

Backend (Python)

Package	Version	Role
FastAPI	0.115+	REST API + WebSocket server
ARQ	0.26+	Async Redis job queue — avoids MPS GPU context fork issues
SQLModel	0.0.21+	ORM — single model for SQLite (local) + PostgreSQL (cloud)
Alembic	1.13+	Database migrations
redis[asyncio]	5.0+	Job queue + PubSub for live pipeline progress events
Pillow + piexif	10+	Thumbnails, EXIF timestamp + focal length extraction

ML Pipeline — Apple Silicon (Metal / MPS)

Tool	Role	Apple Silicon
COLMAP / pycolmap	Camera pose estimation (SfM)	Metal-native via brew install
hloc SuperPoint+SuperGlue	Feature matching for dark/party photos	MPS supported
SAM2 (Meta)	Person masking per photo	MPS with PYTORCH_ENABLE_MPS_F ALLBACK=1
YOLOv8n (ultralytics)	Person bounding boxes to prompt SAM2	MPS supported

Depth Anything v2 Small	Depth hints — Apache 2.0 license	MPS + Core ML
OpenSplat	3D Gaussian Splatting training	Metal-native build required

Frontend (React / TypeScript)

Package	Role
React 18 + Vite 5	SPA — not Next.js (no SSR needed, simpler build)
Three.js 0.169+	3D renderer core
@mkkello/gaussian-splats-3d	Best maintained WebGL Gaussian Splatting renderer for Three.js
Zustand 4.5+	Minimal state management (viewer state, job state, camera)
Tailwind CSS 4 + shadcn/ui	Utility styling + accessible Radix-based components
axios + react-dropzone	File upload with progress events + drag-drop support

ML Pipeline — Full Data Flow

All stages run sequentially. Each stage writes a checkpoint file so failed or cancelled jobs resume from the last completed stage automatically.

Step	Script	Time (300 photos)	Output
1 — Ingest & Validate	01_ingest.py	~5 sec	processed_images/, metadata.json
2 — Temporal Bucketing	02_bucket.py	<1 sec	buckets.json (15-min sliding windows)
3 — COLMAP Full Event	03_colmap_full.py	20–90 min	sparse/0/ — camera poses for all photos
4 — SAM2 Masking	04_mask.py	5–15 min	masks/*.png (255 = exclude people)
5 — Window Sub-Recon	05_window_recon.py	<1 min	windows/{id}/sparse/0/ with anchor photos
6 — OpenSplat Training	06_train_windows.py	~17 min/window	{bucket_id}.ply per time window
7 — Post-Processing	07_postprocess.py	~2 min	scene.json + LOD preview.ply (<5 MB)

Key COLMAP Settings for Party Photos

- **domain_size_pooling=True** — biggest win for dark/low-texture scenes
- **max_num_features=16384** — double default to survive noise and blur
- **camera_mode=PER_IMAGE** — critical for mixed phones from different manufacturers
- **hloc SuperPoint+SuperGlue** — replace SIFT matching for 2–3× more matched pairs

Key OpenSplat Parameters

- **max_gaussians=1,000,000** — hard cap to stay within 24 GB unified memory
- **sh_degree=3** — full spherical harmonics; reduce to 1 if OOM on first attempt
- **iterations=30,000** — approximately 17 minutes on M-series with Metal GPU
- **masks directory enabled** — pass SAM2 mask folder to exclude people from training

Phase-by-Phase Build Plan

Phase 0 — Proof of Concept	Weeks 1–3
• Install pycolmap, OpenSplat, SAM2. Run COLMAP on ETH3D dataset. Benchmark every stage.	
Phase 1 — Single-Event Pipeline	Weeks 4–8
• Build pipeline.py: COLMAP → SAM2 → OpenSplat as subprocess chain with per-stage logging.	
Phase 2 — Browser 3D Viewer	Weeks 9–13
• React + Vite + Three.js scaffold. gaussian-splats-3d renderer. Load static .ply.	
Phase 3 — Timeline Layer ■ Highest Risk	Weeks 14–20 + 4-week buffer
• Week 14: Research sprint ONLY — read CSS paper (arXiv 2409.08562) + WildGaussians. No code.	
Phase 4 — Photo Collection UX	Weeks 21–26
• Event creation UI + shareable URL + QR code generation.	
Phase 5 — MVP Polish & Real Event Test	Weeks 27–32
• WebSocket live progress UI showing all 6 pipeline stages. Email notifications.	
Phase 6 — Production Quality	Weeks 33–40
• SQLite → PostgreSQL migration (Alembic). Docker container.	

Milestones Summary

#	Milestone	Target Week	Key Success Criteria
M0	Environment working	3	All tools install; real benchmark times documented on your Mac mini
M1	First reconstruction	2–3	Your room photos → viewable .ply file in Supersplat
M2	Pipeline API complete	8	curl upload → .ply via API; pipeline handles errors without crashing
M3	Browser viewer	13	3DGS scene navigable in Chrome + Safari + Android Chrome
M4	Timeline scrubber	20 ±4	Scrubbing the timeline causes visible changes in the 3D scene

M5	Multi-contributor UX	26	3 non-developers successfully upload photos without your help
M6	MVP at real event	32	Real guests open result; no manual intervention required
M7	Production quality	40	Cloud deployed; strangers can use it without your assistance

Critical Risks & Mitigations

Risk	P	I	Mitigation & Contingency
COLMAP fails on dark/blurry party photos	High	Critical	Blur rejection gate (Laplacian < 50), hloc SuperGlue matcher, domain_size_pooling=True. Contingency: show coverage heatmap, prompt for more photos.
OpenSplat exceeds 24 GB unified memory	High	Critical	Hard cap max_gaussians=1M, memory monitor wrapper (alert at 18 GB, kill at 22 GB), auto-retry with sh_degree=1.
OpenSplat Metal kernel not working on your Mac	High	Critical	Validate in Week 1 before Phase 1. Expected: >2 iter/sec with Metal. Fallback: gsplat MPS community fork.
Phone clock drift makes timeline nonsensical	High	Critical	Visual anchor alignment across devices. Manual per-device offset slider. Contingency: disable timeline, static scene.
Concurrent jobs OOM each other (no queue)	High	Critical	ARQ single-job queue enforced in Phase 1. Never allow parallel GPU-heavy pipeline stages.
People ghost despite SAM2 masking	High	Major	20px mask dilation, YOLO-prompted SAM2 (not auto-mode), float-Gaussian pruning pass after training.
WhatsApp strips EXIF timestamps	Med	Major	Warn user on upload; offer manual time-range picker. Prompt 'upload originals, not re-shares'.
.ply file too large for mobile browser	Med	Major	LOD preview.ply (<5 MB loads in <2s). Mobile devices get 100K-Gaussian scene by default.
Silent PyTorch MPS→CPU fallback	High	Major	Test with PYTORCH_ENABLE_MPS_FALLBACK=0 in CI — makes fallbacks throw exceptions instead of silently degrading.

Validation Gates

Before COLMAP: ≥30 images after blur filtering · ≤50% rejected for blur · ≥60% have DateTimeOriginal EXIF

Before OpenSplat: ≥70% images registered · ≥25 registered images · ≥5,000 sparse 3D points · reprojection error ≤2px

After OpenSplat: Validation PSNR ≥22 dB · SSIM ≥0.75 · Gaussian count ≤2,000,000 · .ply file <300 MB

Privacy & Legal Requirements

3D Gaussian splats rendering recognizable faces constitute biometric data under GDPR Article 9. These requirements must be implemented BEFORE any public launch.

Requirement	Implementation
Face blurring	InsightFace (MIT) detection + 25px Gaussian blur on texture layer. Auto-applied to all public scenes.
Data retention	Raw photos deleted 24h after reconstruction. COLMAP model: 30 days. Final .ply: 90 days (default).
EXIF GPS stripping	Strip GPS from all uploads on ingest. Retain only DateTimeOriginal and FocalLength for pipeline.
Right to erasure	'Report myself in this scene' button visible to viewers. Take-down within 72 hours of report.
Consent gate	Host confirms all attendees were informed before triggering 3D reconstruction.
GDPR / CCPA	Privacy policy + data subject access request mechanism required before accepting EU/California users.

Library License Safety

Library	License	Commercial Use?
COLMAP / pycolmap	BSD-3-Clause	Yes
SAM2 (Meta)	Apache 2.0	Yes
Depth Anything v2 Small	Apache 2.0	Yes — Small variant only
OpenSplat	AGPLv3	Yes — with AGPL network-service awareness
WildGaussians (reference code)	MIT	Yes
InstantSplat (NVLabs)	NVIDIA NC License	NO — do not use
DUST3R / MAST3R	CC-BY-NC-SA 4.0	NO — do not use
Depth Anything v2 Base / Large	CC-BY-NC-4.0	NO — Small variant only

Key Papers to Read

#	Paper & Authors	Why Essential
1	3D Gaussian Splatting — Kerbl et al., SIGGRAPH 2023. arxiv.org/abs/2308.04079	Foundation — everything builds on this
2	NeRF in the Wild — Martin-Brualla et al., CVPR 2021. arxiv.org/abs/2008.02268	Appearance embeddings for variable lighting — the core technique
3	CSS: Crowd-Sourced Gaussian Splatting — Chen et al., 2024. arxiv.org/abs/2409.08562	YOUR EXACT PROBLEM — read before writing any code

4	WildGaussians — Kulhanek et al., NeurIPS 2024. arxiv.org/abs/2407.08447	MIT license — borrow the appearance embedding implementation
5	DUST3R: Geometric 3D Vision Made Easy — Wang et al., CVPR 2024. arxiv.org/abs/2312.14132	Pose-free pipeline (NC license but required reading)
6	Gaussian in the Dark — 2024. arxiv.org/abs/2408.09130	Dark venue and nightclub reconstruction